Seguridad de archivos mapeados con mmap en moon

Sistemas Operativos | Parcial 3: Seguridad de Archivos en Capa Kernel y mmap

Pablo Manjarres

Objetivo y uso

El parcial agrega a moon un módulo de kernel, `kernel/mmap_guard.ko`, que protege un archivo contra modificaciones y ejecución a través de `mmap()`. También agrega el comando `m_fmap`, que mapea un archivo y muestra cada llamada al sistema.

```
make                                  # ./moon, sin cambios respecto a los parciales anteriores
make -C kernel                        # mmap_guard.ko (Linux, headers del kernel en uso)
sudo insmod kernel/mmap_guard.ko path=/tmp/moon_secret.txt
./moon
m_fmap /tmp/moon_secret.txt r         # lectura compartida: permitida
m_fmap /tmp/moon_secret.txt w         # escritura compartida: mmap -> -1 EACCES
sh tests/run_mmap_guard_tests.sh      # ataques, registro, contadores y medición
```

Modos de `m_fmap`: `r` lectura compartida, `w` escritura compartida, `p` copia privada con escritura, `x` ejecución y `u` lectura compartida ampliada con `mprotect()`.

Parte I: preguntas diagnósticas

1. Flujo de mmap() hasta el primer fallo de página. `mmap()` entra a Ring 0 por `syscall` y llega a `ksys_mmap_pgoff()` y `vm_mmap_pgoff()`. Allí se llama a `security_mmap_file()`, que consulta los LSM (SELinux, AppArmor, BPF). Si aceptan, `do_mmap()` calcula los `vm_flags` y `mmap_region()` crea un `vm_area_struct` con `vm_file`, `vm_pgoff` y `vm_ops` del sistema de archivos. No se crea ninguna PTE: la llamada solo reserva el rango virtual. En el primer acceso la MMU recorre PGD→P4D→PUD→PMD→PTE, encuentra la entrada vacía y lanza `#PF`. `exc_page_fault()` llama a `handle_mm_fault()`, que busca la VMA y verifica que el acceso coincida con `vm_flags`. Luego `do_fault()` llama a `vm_ops->fault` (`filemap_fault`), que trae la página del page cache o la lee del disco, e instala la PTE. Una lectura instala una entrada de solo lectura. Una escritura en mapeo privado copia la página (COW). Desde ese momento el acceso ocurre sin pasar por el kernel. Por eso la defensa debe actuar al crear la VMA o al cambiar sus permisos.

2. MAP_SHARED frente a MAP_PRIVATE. Con `MAP_SHARED`, `do_mmap()` pone `VM_SHARED | VM_MAYSHARE`. Las PTE apuntan a la misma página del page cache que ven `read()` y los demás procesos. Una escritura marca la página como sucia y llega al disco por writeback o `msync()`. Con `MAP_PRIVATE` no hay `VM_SHARED`. La primera escritura provoca COW y el proceso recibe una página anónima propia. Si el descriptor es `O_RDONLY`, el kernel quita `VM_MAYWRITE | VM_SHARED` (`mm/mmap.c:464`). En seguridad, un mapeo compartido escribible persiste y es visible para todos. Un mapeo privado no persiste, pero conserva en memoria del proceso los datos leídos.

3. Ciclo de vida COW. Una página privada empieza compartida con el page cache y protegida contra escritura. La primera escritura produce `#PF`. `do_wp_page()` ve que la VMA permite escribir y copia la página a una anónima nueva. Después actualiza la PTE con escritura y libera la referencia a la original. Si el kernel trata como escribible una página que solo lo es por COW, la escritura llega al page cache del archivo. Este error ocurre cuando no respeta `VM_MAYWRITE` o confunde `FOLL_FORCE` con permiso real. Dirty COW (CVE-2016-5195) aprovechó esta falla. Escribir en `/proc/self/mem` mientras otro hilo descartaba la copia con `MADV_DONTNEED` permitía escribir en archivos de solo lectura, como `/etc/passwd` o binarios SUID. Así se lograba una escalada a root.

4. LKM con hook en la tabla de syscalls frente a LSM. Desde 5.7, `kallsyms_lookup_name` no está exportado y `sys_call_table` es de solo lectura. Cambiar CR0.WP está bloqueado por el pinning de CR0 (`arch/x86/kernel/cpu/common.c`). Interceptar ahí es frágil y no cubre caminos internos ni `io_uring`. Además, rompe con KPTI y CFI. Un LSM con el gancho `mmap_file` (antes `file_mmap`) y `file_mprotect` está en el camino exacto de todas las rutas. Recibe `struct file`, `prot` y `flags` ya validados y es la API soportada. Su desventaja es que debe compilarse dentro del kernel: `security_add_hooks()` es `__init`. La opción usada es intermedia: kretprobes sobre `security_mmap_file()` y `security_file_mprotect()`. Ve lo mismo que un LSM y se carga como módulo, pero depende de que los símbolos se puedan sondear.

5. Cifrado transparente con mmap. El descifrado debe ocurrir debajo del page cache, en el sistema de archivos. fscrypt descifra en `->read_folio` y guarda texto claro en el page cache. Cifra en writeback sin modificar la página

en caché. El control de acceso ocurre al abrir el archivo: sin la clave en el keyring no se obtiene el inode descifrado. Otros procesos no pueden mapear el archivo y otro usuario no puede leer la memoria de otro proceso por el aislamiento de tablas de páginas, `ptrace_may_access` y `/proc/pid/mem`. La memoria física también requiere cifrado: AMD SME/SEV o Intel TME cifran la RAM contra ataques físicos o DMA. Al retirar la clave, fscrypt expulsa las páginas en claro del page cache.

6. msync, fsync y páginas sucias. Una escritura por el puntero solo marca la PTE como sucia. El kernel traslada ese bit a la página (`folio_mark_dirty`) en writeback, `msync(MS_SYNC)`, `munmap` o reclamación. Después la escribe por `->writepages`. `fsync` también recorre las páginas sucias del inode. `page_mkwrite` permite al sistema de archivos intervenir en la primera escritura de cada página limpia, no en cada byte. Un filtro que inspecciona `write()` no ve cambios hechos por mmap. Tampoco ve los bytes cambiados después del primer `page_mkwrite`, porque la PTE sigue escribible hasta el writeback. Por eso el módulo impide crear el mapeo compartido escribible en lugar de inspeccionar escrituras.

7. Auditoría con eBPF. Sin recompilar el kernel, un programa eBPF se puede enganchar con `kprobe`/`fexit` a `security_mmap_file` o `handle_mm_fault`, o con tracepoints como `syscalls:sys_enter_mmap` y `exceptions:page_fault_user`. Puede registrar pid, inode, `prot` y dirección en un ring buffer. El verificador garantiza que el programa no rompa el kernel. Para bloquear se usa BPF LSM (`lsm/mmap_file`), habilitado en el kernel de prueba (`CONFIG_BPF_LSM=y`). Su límite es que solo ve eventos: después del primer fallo, los accesos no generan eventos.

8. O_RDONLY mapeado con PROT_WRITE y MAP_PRIVATE. Sí se permite, porque la escritura nunca llega al archivo: COW crea una copia anónima del proceso. El kernel solo exige `FMODE_WRITE` para `MAP_SHARED` con `PROT_WRITE`. Esto no permite cambiar el archivo, pero el proceso puede alterar en memoria su vista privada. Por ejemplo, puede cambiar una configuración ya cargada. También puede dejar datos sensibles en páginas anónimas que pueden ir a swap. Dirty COW mostró que una falla en COW convierte ese permiso en escritura del archivo. Por eso el módulo permite este caso y la prueba confirma que el archivo no cambia.

9. Carreras entre madvise(MADV_DONTNEED) y escrituras. La corrección de Dirty COW (commit 19be0eaffa3a) agregó `FOLL_COW`. Desde 6.1 ese marcador se eliminó y `can_follow_write_pte()` (`mm/gup.c:785`) solo permite escribir con `FOLL_FORCE` en una página anónima exclusiva (`PageAnonExclusive`) de esa VMA. Así ya no escribe en una página del page cache. `MADV_DONTNEED` toma `mmap_lock` en lectura y bloquea la tabla de páginas. GUP vuelve a resolver el fallo si la PTE desaparece. Desde 6.12, `proc_mem.force_override` (`fs/proc/base.c:126`) también limita escribir en `/proc/pid/mem`. La prueba `dirty_cow_race` ejecuta esa carrera durante un segundo y comprueba que el archivo no cambie.

10. struct page, tablas de páginas y bits de la MMU. Cada marco físico tiene un `struct page` o folio con contador de referencias, mapeos, `mapping` e índice. Esto permite saber si una página es del page cache o anónima. La jerarquía PGD→P4D→PUD→ PMD→PTE traduce cada dirección virtual. Cada nivel tiene bits que la MMU verifica en hardware. R/W impide escribir páginas de solo lectura, U/S impide que Ring 3 acceda a páginas del kernel y NX impide ejecutar páginas de datos. Las extensiones SMEP y SMAP impiden que el kernel ejecute o lea memoria de usuario sin intención. Un archivo mapeado sin `PROT_EXEC` tiene NX, así que código inyectado ahí produce `#PF`. Por eso el módulo también niega `PROT_EXEC`.

Parte II: estrategia con apoyo de IA

Flujo de trabajo

La IA se usó como asistente de un ingeniero, no como fuente de verdad. Cada afirmación sobre el kernel se verificó con el código fuente de la versión probada.

- **Exploración.** La IA comparó caminos de intercepción: tabla de syscalls, LSM, kprobes, eBPF y `vm_ops->fault`. También identificó las funciones por donde pasan todas las rutas. Las firmas y las líneas citadas se verificaron en el árbol de Linux 6.18.
- **Código C.** La IA generó el primer borrador del módulo y de `m_fmap`. Se revisó cada símbolo: que exista, que sea sondeable y que use la API actual (`regs_get_kernel_argument`, `module_param_cb`). Se compiló con `-Wall` y con Kbuild sin advertencias.

- **Pruebas y PoC.** La IA propuso los vectores: mapeo compartido escribible, `MAP_SHARED_VALIDATE`, ampliación con `mprotect`, ejecución y carrera Dirty COW. Cada prueba compara el archivo con `read()` antes y después. No se acepta solo el código de retorno.

Prompts estructurados

#	Contexto	Rol	Restricción	Resultado esperado
1	Enunciado del parcial y repositorio moon con dos parciales previos	Arquitecto de kernel Linux	Solución simple, compatible con 5.x/6.x, módulo cargable y sin parchear el kernel	Comparación de puntos de intercepción y elección justificada
2	Kernel 6.18 WSL2, `CONFIG_KRETPROBES=y`, `CONFIG_MODVERSIONS=y`	Programador de módulos	Usar solo símbolos sondeables o exportados, sin CR0, sin `sys_call_table` y sin advertencias	`mmap_guard.c` con política, registro y contadores
3	Contrato `ui_trace_*` de moon y plantilla de `cat_data.c`	Desarrollador del shell	Mismo estilo de los comandos existentes y compilación en Linux y macOS	Comando `m_fmap` con traza de cada syscall
4	Política: denegar escritura compartida y ejecución	Analista de pruebas ofensivas	Usuario sin privilegios con permiso POSIX de escritura y verificación por contenido	`mmap_attacks.c`, carrera Dirty COW y prueba sin módulo como control
5	Resultados de la medición	Revisor de rendimiento	Medir solo lo que se puede reproducir en la máquina	Microbenchmark mmap+fallo+munmap con y sin módulo

Alucinaciones detectadas y corrección

Sugerencia	Problema	Verificación
Sobrescribir `sys_call_table` obtenida con `kallsyms_lookup_name()`	No está exportada desde 5.7	`kernel/kallsyms.c`: no tiene `EXPORT_SYMBOL`
Desactivar la protección con `write_cr0(read_cr0() & ~X86_CR0_WP)`	El kernel fija CR0.WP y lo restaura. Además, es inseguro con varias CPU	`arch/x86/kernel/cpu/common.c:422`
Registrar un LSM desde el módulo con `security_add_hooks()`	Es `__init`; los LSM se compilan dentro del kernel	`security/security.c:629`
Gancho `file_mmap` con firma `(file, reqprot, prot, flags, addr, addr_only)`	Es la firma de 3.x. Ahora es `mmap_file(file, reqprot, prot, flags)`, y el wrapper sondeado recibe `(file, prot, flags)`	`include/linux/lsm_hook_defs.h:202`, `security/security.c:3132`
Usar `VM_MAYSHARE` para decidir si `mprotect` escribe en el archivo	Un `MAP_SHARED` sobre `O_RDONLY` conserva `VM_MAYSHARE` sin `VM_SHARED`. Se usa `VM_SHARED`	`mm/mmap.c:464`

Validación manual

1. Se descargó el árbol `linux-msft-wsl-6.18.54.1` del kernel en ejecución. Se aplicó su `/proc/config.gz` y se compiló para obtener `Module.symvers`, porque `CONFIG_MODVERSIONS=y`.
2. Se buscó con `grep` cada símbolo, firma y bandera usado en el módulo.
3. Se compiló el módulo sin advertencias y se cargó y descargó con `insmod`/`rmmod`. Se verificó `dmesg` en ambos casos.
4. Se ejecutaron los ataques sin el módulo como grupo de control y con el módulo.

Arquitectura

```
Ring 3   proceso: mmap(fd, PROT_WRITE, MAP_SHARED)      mprotect(addr, PROT_WRITE)
------------------------- syscall ------------------------------------------------
Ring 0   vm_mmap_pgoff()                                 do_mprotect_pkey()
             |                                                 |
         security_mmap_file()  <-- kretprobe -->  security_file_mprotect()
             |        mmap_guard: ¿inode protegido?          |
             |        sí + (escritura compartida | exec) -> -EACCES
             |        no -> 0 (solo una comparación de puntero)
         do_mmap() -> VMA                        mprotect_fixup()
             |
         #PF -> handle_mm_fault() -> filemap_fault() -> PTE   (sin cambios)
```

Matriz de compensaciones

Opción	Latencia de mmap/fallos	Cobertura	Cargable sin recompilar	Riesgo
`LD_PRELOAD`	Ninguna	Nula: se evita con syscall directa	Sí	Bajo
Hook en `sys_call_table`	Baja	Parcial: no cubre caminos internos	No en 6.x	Alto: bloqueado por CR0/CFI
LSM compilado	Mínima	Total en creación y cambio de permisos	No	Bajo
BPF LSM	Mínima	Igual a LSM	Sí, requiere `bpf` en `lsm=`	Bajo
kretprobe en ganchos LSM (elegida)	Baja: una comparación por mmap	Igual a LSM para mmap/mprotect	Sí	Medio: depende de símbolos sondeables
Instrumentar `handle_mm_fault`	Alta: cada fallo	Por página, solo el primer acceso	Sí	Alto

Parte III: implementación y argumentación

Código

- `kernel/mmap_guard.c` resuelve el inode de `path` al cargar y registra dos kretprobes. El manejador de entrada compara el inode. Si no coincide, devuelve 1 y no se ejecuta el de retorno. Para el archivo protegido niega `PROT_EXEC` y `PROT_WRITE` con mapeo compartido, y permite lo demás. El manejador de retorno cambia el 0 del gancho por `-EACCES` antes de crear la VMA o modificarla. Cada decisión se registra en `dmesg` con pid, uid, comm y `prot`, y se cuenta en `/sys/module/mmap_guard/parameters/{allowed,denied}`.
- `kernel/Makefile` usa Kbuild. El `Makefile` principal sigue compilando moon igual.
- `cat_memory.c` contiene `m_fmap`.
- `tests/mmap_attacks.c` prueba los ataques. `tests/mmap_bench.c` mide el costo y `tests/run_mmap_guard_tests.sh` ejecuta todo.

Por qué en el kernel

Después del primer fallo, un acceso por puntero no ejecuta ninguna instrucción del kernel. Una biblioteca en espacio de usuario solo puede envolver `mmap()`, y cualquier binario estático o `syscall(SYS_mmap)` la evita. Los ganchos LSM están en el único punto obligatorio: crear la VMA o ampliar sus permisos. Negar ahí impide que se instale una PTE escribible. Por eso no hace falta inspeccionar cada escritura, que sería imposible a la velocidad del hardware.

Resultados

`KDIR=... KCC=gcc-13 sh tests/run_mmap_guard_tests.sh` se ejecutó en Ubuntu 26.04 sobre WSL2, con kernel 6.18.54.1-microsoft-standard-WSL2 y 6 núcleos. El árbol del kernel se compiló con gcc 13, el mismo compilador del kernel en ejecución. Con gcc 15, `insmod` rechazó el módulo porque `struct module` tenía otro tamaño.

Las 18 comprobaciones pasaron. El módulo compila sin advertencias. Sin el módulo, el mismo usuario sí puede crear el mapeo compartido escribible: es el grupo de control. Con el módulo:

- Se permiten la lectura compartida, la copia privada y la lectura compartida sobre un descriptor `O_RDWR`. Los bytes mapeados coinciden con `read()`.

- Se rechazan con `EACCES` la escritura compartida (`MAP_SHARED` y `MAP_SHARED_VALIDATE`), la ejecución y las ampliaciones con `mprotect` a escritura o ejecución.
- En la carrera Dirty COW, el kernel aceptó 153 736 escrituras por `/proc/self/mem`. Todas cayeron en la copia privada y el archivo quedó idéntico byte a byte.
- `m_fmap w` muestra `mmap -> -1 EACCES`. `dmesg` registra cada decisión y los contadores terminaron en `allowed=6 denied=6`.
- Al descargar el módulo, la escritura compartida vuelve a funcionar.

Medición con 200000 iteraciones de `mmap` de una página, primer acceso y `munmap`:

Archivo	Sin módulo (ns, mediana de 5)	Con módulo (ns, mediana de 5)	Diferencia
Protegido (entrada completa del probe)	8560	8445	-1.3 %
Otro archivo (solo comparación de inode)	8809	8976	+1.9 %

Cada iteración produjo un fallo de página menor, igual con y sin módulo. Por lo tanto, el módulo no cambia la tasa de fallos: actúa antes de crear la VMA, no en `handle_mm_fault`. Las diferencias son menores que la variación entre corridas, de 7771 a 8883 ns. En esta máquina, el sobrecosto no se distingue del ruido. Se midió con `clock_gettime` y `getrusage`, porque el kernel de WSL no trae `perf` ni `strace`. El costo esperado es el de un kprobe por `mmap` y `mprotect`; las lecturas y escrituras por puntero no tienen costo adicional.

Limitaciones y trabajo futuro

- **Escrituras normales.** El módulo protege mmap. Las escrituras con `write()` siguen sujetas a los permisos POSIX. Un tercer gancho en `security_file_permission` cerraría ese camino a costa de una comparación por cada `read`/`write` del sistema.
- **Mapeos previos.** Un mapeo creado antes de cargar el módulo sigue vigente.
- **Root.** Root puede descargar el módulo. Con `lockdown=integrity` y módulos firmados, o con el mismo código como BPF LSM fijado al arranque, se evita.
- **kprobes.** Se pueden desactivar o pueden cambiar los nombres de los símbolos. Un LSM compilado o BPF LSM es más estable.
- **Un archivo.** Se protege un solo archivo, identificado por inode. Si se renombra el archivo, sigue protegido. Si se reemplaza por otro inode, no.
- **Memoria física.** Un proceso con acceso a la RAM, DMA o un hipervisor comprometido ve los datos en claro. Mitigaciones: IOMMU para DMA, Intel TME/AMD SME para cifrar la RAM y SEV-SNP/TDX para aislar la VM del hipervisor.